

Todo 상태 전이와 Domain Event

이전 구현과 개선 구현을 실제 코드 흐름으로 비교하며,
값의 관계가 왜 domain action에서 함께 결정되어야 하는지 정리한 리뷰

핵심 결론

Todo가 생성, 완료, 삭제되었다는 하나의 domain 사실이다. 따라서 변경된 Todo와 그 사실을 나타내는 event는 같은 순수 transition에서 함께 만들어지고, program은 둘을 atomic하게 저장하는 역할을 맡는다.

대상 코드: `assignments/effect-todo-server`

1. 먼저 구분할 것

이 문서는 event를 만드는 코드가 파일 하나에 있느냐만 보지 않는다. 더 중요한 질문은 누가 Todo의 새 상태와 event의 의미를 결정하는가이다.

질문	이전 구조	개선 구조
완료된 Todo는 어떤 값인가?	repository의 UPDATE/row mapper가 만듦	domain transition이 만듦
완료 event는 어떤 값인가?	program이 별도 조립	같은 transition이 Todo에서 파생
DB와 audit 저장은 누가 하는가?	program과 repository	program이 AtomicRunner 안에서 조합

정확한 표현

Create는 이전에도 program이 Todo와 event를 모두 만들었다. 그러나 Complete/Delete는 실제 변경 Todo를 repository가 만들고 event를 program이 만들었다. 따라서 '서로 다른 레이어' 문제는 특히 Complete/Delete에서 명확했다.

2. Complete: 이전 코드와 개선 코드

이전 program은 repository 결과와 바깥 시간 값을 섞어 event를 만들었다.

```
// done-todo.ts
const completedTodo = yield* repository.markDone(id, completedAtMillis);

yield* eventStore.append(
  makeTodoCompletedEvent({
    todoId: completedTodo.id,
    occurredAtMillis: completedAtMillis,
  }),
);
```

이 코드가 전제하는 관계는 다음과 같다.

```
repository.markDone(id, completedAtMillis)
-> DB completed_at_millis = completedAtMillis
-> completedTodo.completedAtMillis completedAtMillis

event.occurredAtMillis = completedAtMillis
```

현재 SQLite에서는 이 전제가 맞는다. 하지만 event는 repository가 실제로 저장한 Todo의 시간이 아니라 program이 전달했던 시간에 의존한다. repository 구현, DB trigger, 저장 시간 정책이 바뀌면 두 값의 일치가 깨질 수 있다.

개선 후에는 domain transition이 하나의 지역 Todo에서 event를 만든다.

```
// domain/todo-transition.ts
const completedTodo = makeCompletedTodo(todo, completedAtMillis);

return Either.right({
  todo: completedTodo,
  event: makeTodoCompletedEvent({
    todoId: completedTodo.id,
    occurredAtMillis: completedTodo.completedAtMillis,
  }),
});
```

관계가 코드에 드러나는 방식

event.todoId는 completedTodo.id에서, event.occurredAtMillis는 completedTodo.completedAtMillis에서 직접 나온다. program은 event 필드를 다시 조립하지 않고 change.event를 저장한다.

3. Delete: 왜 titleSnapshot과 시간이 함께 결정되어야 하나

이전 delete program은 repository가 돌려준 Todo와 program의 바깥 시간을 섞었다.

```
// delete-todo.ts
const deletedTodo = yield* repository.delete(id, deletedAtMillis);

yield* eventStore.append(
  makeTodoDeletedEvent({
    todoId: deletedTodo.id,
    titleSnapshot: deletedTodo.title,
    occurredAtMillis: deletedAtMillis,
  }),
);
```

개선 후 transition은 삭제 전 Todo와 삭제 시간을 받아, 새 상태와 event를 같은 함수에서 만든다.

```
// domain/todo-transition.ts
const deletedTodo = makeDeletedTodo(todo, deletedAtMillis);

return {
  todo: deletedTodo,
  event: makeTodoDeletedEvent({
    todoId: deletedTodo.id,
    titleSnapshot: deletedTodo.title,
    occurredAtMillis: deletedTodo.deletedAtMillis,
  }),
};
```

여기서 event의 의미는 단순히 'DB row가 update되었다'가 아니다.

```
DeletableTodo(title = "" "" "")
-> DeletedTodo
-> TodoDeletedEvent(titleSnapshot = "" "" "")
```

예시 1. DB 시간이 정본이 되는 경우

나중에 DB가 서버 시간 대신 DB 시간을 사용하도록 바뀌거나, trigger가 저장 시간을 보정할 수 있다. 이전 구조는 event가 program의 시간만 참조하므로 DB가 실제 저장한 시간과 갈라진다.

```
program deletedAtMillis:      1000
DB stored deleted_at_millis:  1005
old event.occurredAtMillis:    1000

problem: audit says 1000, persisted Todo says 1005
```

개선 구조는 domain이 선택한 삭제 시각으로 Todo와 event를 동시에 만든다. 만약 DB 시간이 정본이라는 요구가 생긴다면, 시간을 Clock/DB 중 어디서 결정할지와 event 의미를 한 정책으로 다시 결정해야 한다. 그 결정이 program의 흩어진 event 조립 코드에 숨어 있지 않다는 것이 핵심이다.

예시 2. 삭제 시 title을 변환하거나 삭제하는 경우

개인정보 보호나 retention 정책 때문에 persistence가 삭제 후 title을 '(deleted)'로 바꿀 수 있다. 이때 event의 titleSnapshot이 무엇을 뜻하는지 business decision이 필요하다.

```
delete 00 domain title:      "Call customer Kim"
00 0 DB title:              "[deleted]"

000 00 A: event0 00 000 00 000 000
000 00 B: event0 DB0 000 00 0 000 000
```

이전 구조에서는 repository가 반환한 값에 따라 event 값이 우연히 결정될 수 있다. 개선 구조는 deleteTodo가 입력 Todo.title을 snapshot으로 사용하므로 의미 A를 명시적으로 선택한다.

예시 3. repository 구현을 바꾸는 경우

SQLite, file, memory repository가 서로 시간 정밀도, 정규화, trigger 유무가 다를 수 있다. 이전 구조에서는 program이 repository 구현의 반환 모양과 바깥 변수를 조합해 event를 만들었다. 따라서 repository 교체가 event 의미까지 흔들 수 있었다.

개선의 요지

domain transition은 persistence 구현과 무관한 'Todo가 삭제되었다'는 사실을 만든다. repository는 그 Todo를 조건부로 저장할 수 있는지 확인할 뿐, event의 의미를 결정하지 않는다.

4. 왜 이것을 domain event로 볼 수 있는가

값을 함께 묶었다고 자동으로 domain event가 되는 것은 아니다. 다음 질문으로 판단한다.

질문	TodoDeletedEvent의 답
SQL table을 알아야 성립하는가?	아니오. Todo가 삭제되었다는 사실은 DB 없이 성립한다.
requestId/actorId를 알아야 성립하는가?	아니오. 그것들은 audit context다.
업무 언어로 말할 수 있는가?	예. 'Todo가 완료되었다', 'Todo가 삭제되었다'.
다른 정책이 반응할 수 있는가?	가능하다. notification, projection, 통계, audit가 반응할 수 있다.

반대로 아래 값은 domain event에 넣지 않는다.

```
requestId, actorId, audit_logs table name, SQL action name

-> event store / adapter0 RequestContext0 00 audit row0 0000.
```

현재 SQLite event store는 받은 event를 만들지 않고, event.todoId와 event.occurredAtMillis를 audit row에 저장한다. requestId/actorId만 RequestContext에서 붙인다. 이것이 domain fact와 operational audit context의 경계다.

5. AtomicRunner 안에서의 실제 흐름

```
repository.find(id)
-> completeTodo(currentTodo, time)
-> { todo: CompletedTodo, event: TodoCompletedEvent }
-> repository.saveCompletedIfActive(todo)
-> eventStore.append(event)
```

SQLite의 저장은 여전히 조건부 update다.

```
UPDATE todos
SET status = 'completed', completed_at_millis = ?
WHERE id = ? AND status = 'active'
RETURNING *
```

그래서 두 요청이 동시에 Active Todo를 읽어도 event는 하나만 저장된다.

```
A: find Active -> transition -> conditional save succeeds -> append Event A
B: find Active -> transition -> conditional save returns 0 rows
   -> TodoAlreadyCompleted -> Event B is never appended
```

중요한 구분

domain event를 '만드는 것'은 메모리에 값이 생기는 일이다. event를 audit store에 저장하는 일은 AtomicRunner 안에서 Todo 저장 성공 뒤에만 일어난다.

6. TypeScript가 보장하는 것과 테스트가 보완하는 것

TypeScript는 Todold와 EpochMillis가 맞는 종류의 값인지 확인한다. 그러나 두 값이 반드시 동일하다는 dependent type 증명까지 제공하지는 않는다.

보장 수단	현재 코드에서 보장하는 내용
구조	transition 하나가 { todo, event }를 같이 반환한다.
구현	event 필드가 transition 안의 Todo 필드에서 직접 파생된다.
타입	Todold, TodoTitle, EpochMillis 같은 값의 종류를 유지한다.
테스트	event.todold와 todo.id, event 시간과 Todo 시간을 비교해 회귀를 막는다.

권장하는 직접 관계 테스트는 아래와 같다.

```
expect(change.event.todoId).toBe(change.todo.id);
expect(change.event.occurredAtMillis).toBe(
  change.todo.completedAtMillis,
);
```

현재 test/todo-transition.test.ts는 Todo와 event의 값을 각각 확인한다. 위처럼 두 값을 직접 비교하는 assertion을 추가하면 이 문서에서 말한 관계가 테스트에도 더 선명하게 드러난다.

7. 최종 체크리스트

- 이 event는 DB 저장 성공 자체가 아니라, 업무적으로 의미 있는 상태 변화를 말하는가?
- event의 값은 transition이 만든 Todo에서 직접 파생되는가?
- requestId, actorId, SQL 정보 같은 audit context를 domain event와 분리했는가?
- 조건부 저장 실패 시 event가 외부에 append되지 않는가?
- Todo와 event의 ID/시간 관계를 순수 domain test로 비교하는가?